the first iteration.

#4 Create a new hash table to keep track of the distance of each vertex from vertex `start`, i.e. the sum of the weight of edges that needs to be traversed to get from vertex `start` to each other vertex.

#5 Also create another hash table to keep track, for each vertex `u`, of the vertex through which `u` was reached. This dictionary can later be used to reconstruct the path from `start` to the goal.

#6 While we initialized the vertex distances to infinity (or, equivalently, to the largest value that can be stored) for most vertices, for the starting point only, we need to set its distance (from itself) to 0.

#7 Start a loop, running until the priority queue is empty. It will run at least once, because of line #3.

#8 Extract the top of the priority queue. The vertex extracted will become current vertex for this iteration, `v`.

#9 If we reached the goal, we are done, just return current vertex. The function `isGoal` can abstract the condition checked to find the goal: it can be reaching one or more specific target vertices, or getting to vertices that satisfy a certain condition.

#10 Iterate over the outgoing edges for current vertex.

#11 If vertex `u` hasn't been visited yet, then maybe its distance from start can be smaller by passing through `v`.

#12 If so, we need to update its distance, its parent, and then add update the queue, so that `u` will be popped from it at the right time (i.e., when it becomes the closest vertex to the frontier of visited vertices).

#13 If the goal is never reached, we need to return `null` or, equivalently, another value that signals that search failed.